CIS 310

Computer Organization and Assembly Language

Section 001 · Fall 2026 · 4 credit hours

Active Fall 2026 syllabus — use Canvas for announcements, released assignment details, and submission.

Instructor	Dr. Probir Roy
Email	probirr@umich.edu
Office	CIS Building, Room 230
Phone	313-583-6620
GSI or grader	None currently assigned or confirmed; check Canvas and department announcements for any future staffing update
Class meetings	Mondays and Wednesdays, 10:00–11:45 a.m., ELB 1329
Delivery format	In-person lecture; first meeting Wednesday, August 26, 2026
Instructor office hours	Mondays and Wednesdays, 9:30–10:00 a.m. and noon–1:00 p.m., CIS Building, Room 230; or by appointment
Course site	Fall 2026 CIS 310 Canvas
Calendar	official 2026–2027 academic calendar

Source-of-truth rule. This syllabus states the Fall 2026 course structure, grading framework, and course policies. Canvas provides released assignment details, due dates, required files, announcements, and submission receipts. A written instructor correction or current Canvas announcement may clarify or update tentative logistics; ask when two sources appear inconsistent.

Course description and prerequisites

The architecture of computer systems and associated software. Topics include digital logic circuits, computer interfacing, interrupt systems, input/output systems, memory systems, assemblers and assembly-language programming, and computer networks. This is the current UM-Dearborn catalog description.

Prerequisites: MATH 115, CIS 200, and CIS 275. Students should be ready to use binary and hexadecimal notation, Boolean reasoning, and basic programming concepts. If any prerequisite topic feels unfamiliar, use the self-paced preparation and practice path in SystemStudio and seek help early.

Dearborn Discovery Core: Not applicable; CIS 310 is a required course in the Computer and Information Science major.

Program learning goals

The course supports the B.S. in Computer and Information Science program goals. Graduates should be able to analyze complex computing problems; design, implement, and evaluate computing solutions; communicate effectively; recognize professional, ethical, legal, and social responsibilities; function effectively on teams; and apply computer-science theory and software-development fundamentals. CIS 310 most directly develops problem analysis, computing-solution design/evaluation, technical communication, teamwork, and application of computing fundamentals.

Learning outcomes

By the end of the course, a successful student should be able to:

1. represent and interpret signed and unsigned data in binary and hexadecimal;

2. analyze and synthesize combinational circuits using truth tables, Boolean algebra, Karnaugh maps, adders, decoders, and multiplexers;
3. analyze and build sequential circuits using latches, flip-flops, registers, counters, and finite-state behavior;
4. explain memory organization, buses, address decoding, caches, I/O, interrupts, and the instruction cycle;
5. describe register-transfer operations, datapath/control interaction, and basic processor pipelining;
6. write, trace, and explain introductory IA-32 assembly programs, including registers, flags, memory, stack, branches, and procedures; and
7. integrate and test processor components, then communicate expected behavior, observed evidence, and remaining uncertainty.

Learning workflow and course technology

Before each topic, use SystemStudio's self-paced sequence **accessible HTML lecture → read → watch → retrieve → build or trace**: begin with the primary structured HTML lecture, continue with focused open-text sections and transcript-checked author videos, and try five distinct questions without reopening the sources. Continue through all eight questions for the module's confidence set. The untagged lecture PDF is an optional visual archive, not the primary lecture format. When a required hands-on lab is mapped, complete **predict → build or step → inspect evidence → explain**. Formative circuit labs create fresh files and open them in the complete upstream Digital application. Assembly activities distinguish actual assembler/linker/executable evidence from a separately labeled instruction-trace visualization. Neither supplies a graded assignment artifact. Bring one unresolved point to class. Class meetings then combine explanation, prediction, worked examples, circuit or assembly demonstrations, and short evidence checks. For project work, use **predict → build → test → inspect evidence → explain**.

Canvas	The authoritative course site and the only submission destination. Confirm every due date and submission receipt there.
SystemStudio CIS 310	The VS Code extension maps all 13 lecture topics to primary accessible HTML lectures, focused open-book readings, transcript-checked author videos, eight-question confidence sets, and hands-on work where appropriate. It also provides Full Digital, seven guided circuit builds, actual assembly-toolchain routing, five instruction-trace activities, quiz mode, confidence checks, explanation and justification, review, the syllabus, calendar, 13 optional offline visual PDF archives, seven coursework/final-project references, Assignment Mission Control, setup checks, a guided tutorial, and a local help router.
Digital v0.31	The extension installs and checksum-verifies the complete upstream application. Linux hosts transport its actual Swing desktop into the VS Code tab. Windows and macOS use an extension-managed Docker Desktop runtime to stream the same application into the tab; its native window is an explicit fallback. Java 8 or newer is required on Linux and for native/CLI use; the Windows/macOS container supplies Java. The read-only preview and testcase runner also use Digital's official tools.
Real assembly toolchains	Linux NASM activities invoke actual NASM, GNU `ld`, and the resulting ELF32 executable. Exact MASM/Irvine32 activities require Windows, Microsoft `ml.exe`/`link.exe`, and the official Irvine library. SystemStudio does not emulate or relabel MASM on other systems.
Instruction Trace Tutor	A separately labeled, bounded source-level visualization supports prediction practice with registers, flags, memory, stack, virtual input/output, and control flow. It is not MASM or NASM, emits no machine code, and cannot establish that source assembled successfully.
Student computer	A current desktop version of VS Code and enough access to install the course extension. Contact the instructor promptly if hardware, Java, accessibility, or installation creates a barrier.

The Learning Center's reading/video checkmarks, retrieval practice, guided-lab checkmarks, coursework status, and final-project self-evaluation are ungraded and self-reported. Its local dashboards describe preparation and practice evidence; they do not certify mastery or send learning history to Canvas or the instructor. A visually separate manual grade estimator applies the published category weights and two-lowest participation-quiz drop rule only to scores a

student enters from Canvas. Its result is a planning estimate, not an instructor evaluation or official Canvas grade. Use the mapped sources and visible evidence to decide what to review, then ask for human help when uncertainty remains. External PDFs and videos may open at their beginning rather than at an exact page or timestamp.

Technology recovery plan

If a tool fails, save the source file, capture the exact error and relevant screenshot, and report the expected result, observed result, evidence, and one attempted fix. Continue reading or prediction work while the issue is resolved. Do not wait until the submission deadline to report a setup problem. Canvas remains available independently of the extension.

Course materials

The extension bundles the presentation sequence locally as PDFs so students do not depend on external file-hosting access during study. The course pack also separates the three homework references from the three processor-project milestones:

- **Homework 1:** logic foundations;
- **Homework 2:** sequential logic and state machines;
- **Homework 3:** memory and assembly foundations;
- **Project 1:** 4-bit registers/data memory and 8-bit instruction storage;
- **Project 2:** register file and ALU; and
- **Project 3:** integrated 4-bit processor.

Required open text: David Tarnoff, *Computer Organization and Design Fundamentals*. Chapter PDFs are available without charge from the author's official book page. SystemStudio links to the relevant chapter sections before each lecture; it does not redistribute the book.

Required assembly reference: Kip R. Irvine, *Assembly Language for x86 Processors*, any recent edition. Use Canvas for any updated edition or access instructions.

The mapped companion videos come from Tarnoff's official author channel or the ETSU open educational-resource series. The videos reinforce, but do not replace, the assigned reading. External book and video sites have their own privacy practices; the accessible HTML lectures and optional visual PDF archives remain available locally.

Assessment, grading, and submission

Assessment category	Course weight
Participation quizzes and in-class evidence checks	15%
Three written homework assignments and three implementation assignments/processor milestones	65%
Final processor project and demonstration	20%
Total	**100%**

Canvas provides the released questions, point values within each category, due dates, required files, and submission format. The three implementation assignments form one cumulative processor project: 4-bit registers/data memory plus 8-bit instruction storage, the 4-bit register file/ALU, and the integrated 4-bit processor. The instruction word is 8 bits so it can hold opcode and operand fields; the general-purpose registers, ALU data, and data-memory words remain 4 bits. During the final presentation, students demonstrate and explain that **same cumulative 4-bit processor and its instructional-ISA program**; it is not a separate processor redesign. The required evidence, presentation format, and submission details will appear in Canvas.

Before uploading a circuit, students may run SystemStudio's local public preflight suites against their own `.dig` files. The suites exercise the published top-level interfaces for the 4-bit register and PC, 8-bit instruction register

and 16-address instruction memory, 16-address × 4-bit data memory, four-register × 4-bit register file, and all 2,048 required ALU input combinations. A 25-vector integrated suite then runs the published six-instruction program through the four-state fetch/decode/execute/writeback controller, including register and data-memory behavior. These checks are formative and remain on the student's computer; passing does not constitute submission, predict a score, or replace the instructor's evaluation of design, student-created tests, documentation, explanation, or other rubric criteria.

Letter-grade scale

Final weighted percentages are converted using the following scale. Boundaries are applied to the unrounded course percentage; Canvas display rounding does not move a grade across a boundary.

A+	≥ 96.5	A	94.0–$<$ 96.5	A−	90.0–$<$ 94.0
B+	87.0–$<$ 90.0	B	84.0–$<$ 87.0	B−	80.0–$<$ 84.0
C+	77.0–$<$ 80.0	C	74.0–$<$ 77.0	C−	70.0–$<$ 74.0
D+	67.0–$<$ 70.0	D	64.0–$<$ 67.0	D−	60.0–$<$ 64.0
E	$<$ 60.0				

Course-specific assessment policies

- **Participation quizzes:** Complete participation quizzes and in-class evidence checks during the class meeting in which they are assigned. Late participation work is not accepted. The two lowest quiz scores are dropped when the participation category is calculated.
- **Homework:** Written homework is individual work unless the released Canvas instructions explicitly authorize collaboration. Late homework is accepted with a 10% deduction from the earned score.
- **Implementation assignments:** These may be completed individually or in an instructor-authorized team of no more than two students. Each student must be able to explain the submitted design and evidence. Late implementation work is accepted with a 10% deduction from the earned score.
- **Final cumulative 4-bit processor presentation and demonstration:** The final assessment presents the same 4-bit processor developed through the three implementation assignments, its 8-bit instructional-ISA program, test evidence, and a live presentation/demonstration during final examination week. It may be completed individually or in an instructor-authorized team of no more than two students. Every team member must participate in and explain the processor, program encoding, test evidence, and demonstration. Any separately assigned IA-32 MASM/NASM work is a different toolchain artifact and does not execute on the instructional processor. The exact date, time, room, required artifacts, and presentation format will be announced in Canvas. The final deliverable and demonstration are not accepted late except when the instructor approves an accommodation or documented extenuating circumstance.
- **Make-up work:** Contact the instructor as soon as possible about a serious circumstance affecting an assessment. Make-up arrangements are not automatic and must be approved by the instructor consistently with university policy and any approved accommodation. Do not email sensitive medical documentation; ask where it should be sent.

Submission rule: Submit every required deliverable in Canvas and verify that Canvas recorded it. A local file, simulator run, email attachment, or SystemStudio preview is not a course submission unless the current Canvas assignment explicitly says otherwise.

Final examination/project demonstration: The cumulative 4-bit processor and assembly-program presentation/demonstration will take place during Fall 2026 final examination week. The **exact date, time, room, presentation order, and submission deadline are to be announced in Canvas**. Students should keep the examination period available until the official course announcement is posted.

Communication, feedback, and help

- Use Canvas Inbox or university email for private matters. Use the designated Canvas discussion for questions whose answers may help the class; do not post grades, accommodations, or other private information there.

- Before asking about a technical problem, record: what you expected, what happened, the exact evidence, what you tried, and the decision you need help making.
- The SystemStudio local FAQ chat can route a question to a topic, tool, syllabus, calendar, or Canvas. It is deterministic and does not send the conversation to an AI service; it does not know private grades, live deadlines, or instructor decisions.
- The optional U-M Maizey course tutor opens in Canvas with the student's own U-M authentication. Use it for source-grounded hints and explanations, then verify the cited course source. The extension does not contain the instructor's personal LLM credential.
- Use **Ask a Question Before Class** to structure a concept or complex-question request for the Canvas discussion. Canvas controls whether the final post is named or anonymous; if Canvas does not display an anonymous choice, do not assume the post is anonymous.
- Seek help early when a prerequisite, setup step, circuit behavior, assembly trace, assignment instruction, or team interaction is unclear.

Academic integrity, collaboration, and AI

Students are responsible for following the UM-Dearborn academic-integrity requirements, the Academic Code of Conduct, and the specific rules on each Canvas assignment. Do not submit another person's work, share solution artifacts, or misrepresent generated or copied work as your own.

Collaboration and generative-AI permissions are assignment-specific. If a Canvas assignment does not clearly authorize a type of assistance, ask the instructor before using it in submitted work. For an ungraded SystemStudio readiness or tutorial question, commit to an answer and one reason before requesting an AI hint or critique; asking for the answer first defeats the retrieval activity. The U-M Maizey course tutor may support formative learning through incremental hints, analogous examples, source links, and checks of student-supplied reasoning. For homework, projects, quizzes, exams, reports, and other graded work, it must not be treated as an answer key or used to produce a final answer, finished circuit, complete program, report prose, or other submission-ready artifact. AI output can be inaccurate; students remain responsible for checking it against course evidence and for the work they submit. The SystemStudio checkpoint and tutor prompt cannot control a different AI website or guarantee model compliance. The SystemStudio workflow may organize instructor-approved materials and explain visible tool evidence, but it does not replace the student's reasoning or certify that an answer is correct.

If an assignment permits generative AI, preserve the prompts and output needed to document the assistance, cite or acknowledge it in the format required by that assignment, and be prepared to explain every submitted component. Unauthorized generation, paraphrasing, debugging, or solution production may violate the assignment rules and academic-integrity requirements even when the output is edited afterward.

Attendance and classroom conduct

Regular, on-time attendance is expected because explanation, prediction, participation, demonstrations, and feedback occur during class. Attendance itself is not a substitute for participation. Students are responsible for announcements and material covered during an absence and should use Canvas and a classmate to identify what was missed. Notify the instructor promptly when a university-approved activity, religious observance, documented circumstance, or approved accommodation affects participation; the applicable university policy governs available arrangements.

Contribute to a respectful learning environment. Ask questions, critique designs rather than people, protect private information, and give collaborators a fair opportunity to contribute and explain the work. Recording or redistributing class material requires instructor permission unless an approved accommodation states otherwise.

Accessibility, well-being, and university policies

Students who encounter disability-related barriers or need academic accommodations should contact Disability and Accessibility Services (DAS) and communicate approved accommodations to the instructor. Course technology should

not prevent equitable participation; report accessibility barriers as early as possible so an alternative path can be arranged.

Students experiencing food insecurity may use the UM-Dearborn Food Pantry, which provides food and personal-care items to currently enrolled students. Consult the Pantry website for current hours, location, eligibility, and appointment or access instructions.

University-wide course policies on attendance, academic integrity, counseling and psychological services, disability/accessibility, safety, and harassment, violence, bias, and discrimination are available through the Canvas **Course Policies** menu and the University's Course Policies and Information for Syllabi page. These policies apply in addition to the course-specific rules above. Student support and well-being resources remain available through UM-Dearborn.

Copyright and permitted use

Course materials are provided to enrolled students for their personal educational use. Do not publish, sell, upload, or redistribute slides, assignment material, recordings, assessment content, or another student's work without the copyright holder's permission. Links to open resources remain governed by their licenses and source-site terms.

Tentative Fall 2026 course calendar

The university dates below are verified against the official Fall 2026 calendar. Topic placement is a proposed instructional sequence and may move; Canvas announcements control changes. No assignment deadline is inferred here.

Mtg.	Date	Planned focus
1	Wed Aug 26	Orientation; abstraction; data representation
2	Mon Aug 31	Binary and hexadecimal representation
3	Wed Sep 2	Signed data, two's complement, and adders
–	Mon Sep 7	**Labor Day holiday — no class**
4	Wed Sep 9	Boolean operations and truth tables
5	Mon Sep 14	Boolean algebra and circuit simplification
6	Wed Sep 16	Karnaugh-map method
7	Mon Sep 21	Karnaugh maps and don't-care conditions
8	Wed Sep 23	Decoders, multiplexers, and data selection
9	Mon Sep 28	Combinational design and evidence-based testing; Homework 1 preparation checkpoint
10	Wed Sep 30	Latches, clocks, and state
11	Mon Oct 5	Flip-flops, registers, and counters
12	Wed Oct 7	Sequential-circuit analysis and verification; Homework 2 / Project 1 preparation checkpoint
13	Mon Oct 12	Memory organization and memory maps
14	Wed Oct 14	Buses and address decoding
15	Mon Oct 19	I/O protocols and polling
16	Wed Oct 21	Interrupts and asynchronous behavior
17	Mon Oct 26	Memory hierarchy and storage latency
18	Wed Oct 28	Cache and locality
19	Mon Nov 2	Register-transfer language and micro-operations
20	Wed Nov 4	Arithmetic units and ALU design
21	Mon Nov 9	Control unit and instruction cycle
22	Wed Nov 11	Datapath/control integration; Project 2 preparation checkpoint
23	Mon Nov 16	Processor components and pipelining
24	Wed Nov 18	Address spaces and x86 registers
–	Nov 21–29	**Thanksgiving recess — no class Nov 23 or Nov 25**
25	Mon Nov 30	Assembly syntax, translation, and execution model
26	Wed Dec 2	Instruction-trace visualization: registers, flags, stack, and memory; Homework 3 / Project 3 preparation checkpoint
27	Mon Dec 7	Integration, review, and last regular class

Mtg.	Date	Planned focus
–	Dec 8–9	Study days
–	Final examination week	Cumulative 4-bit processor and assembly-program presentation/demonstration; exact date, time, room, order, and submission requirements to be announced in Canvas

Keeping course information current

Use this syllabus for the stable course structure, grading framework, and policies, and use the SystemStudio calendar for official term dates. Use Canvas for released assignments, point values within the stated categories, deadlines, submission, announcements, and approved clarifications. A change to a stable syllabus rule will be communicated in writing. When two sources appear inconsistent, follow the newest written instructor clarification and ask before acting.

Source note. Updated August 20, 2026. This document follows the component order of the current UM-Dearborn syllabus template and preserves the applicable structure and course policies of the Fall 2025 CIS 310 syllabus. The current catalog supplies the title, credits, description, and prerequisites; the Fall 2026 class schedule supplies section 001, meeting time, room, and format; and the 2026–2027 academic calendar supplies term and examination-period dates. Canvas is authoritative for the announced CIS 310 final-project demonstration date/time, released assignment details, and submission.